

On This Page >



Exports

⚠ Experimental

This feature is still experimental and will change in the future.

Overview

Exports are a mechanism for services to share structured data.

They provide a way for different machines and service instances to discover and access information they need from each other,

Use Cases

Exports are useful when you need to:

- Share network configuration (IP addresses, ports, etc.) between service components
- Distribute generated credentials or connection strings to consuming machines
- Provide discovery information for peers in a distributed service
- generally share information between machines or services.

Examples:

- A VPN service where peers need to know each other's public endpoints
- A database cluster where replicas need the primary's connection string
- A monitoring system where agents need the metrics collector's address

How Exports Work

Exports use a scope-based system to publish data. Each export is tagged with a scope that identifies its source:

- **Service:** The name of the service (e.g., "zerotier", "wireguard")
- **Instance:** A specific instance of that service
- **Role:** A role within the service (e.g., "client", "server", "peer")
- **Machine:** A specific machine name

Scope levels:

- **Service level:** Data specific to a service across all instances
- **Instance level:** Data specific to a service instance
- **Role level:** Data specific to a role
- **Machine level:** Data specific to a single machine

Important

Exports are stored internally using scope keys, therefore you should always use the `clanLib` helper functions to work with them.

The internal format is subject to change.

Defining Exports in Your Service

To export data from your service, use the `mkExports` helper function available in the `perInstance` or `perMachine` context:

```
1 { clanLib, ... }: {  
2   roles.peer = {  
3     perInstance = { instanceName, mkExports, roles, ... }: {  
4       exports = mkExports {  
5         # Your exported data here  
6         peer.host.plain = "192.192.192.12";  
7       };  
8     }  
9     nixosModule = { ... }: {  
10    };  
11  };  
12 };
```

```
13 }
```

The `mkExports` function automatically creates the appropriate scope for your exports based on the current service, instance, role, and machine context.

Accessing Exports

Within your service module, you can access exports using the `clanLib.exports` helper functions.

`selectExports` - Query Multiple Exports

Get all exports matching specific criteria

```
1 { exports, clanLib, ... }: {
2   perInstance = { ... }: {
3     nixosModule = { ... }: {
4       # Get all exports from the "vpn" service
5       vpnConfigs = clanLib.selectExports
6         { service = "vpn"; }
7       exports;
8     };
9   };
10 }
```

Query parameters (all optional, default to wildcard `"*"`):

- `service` - Filter by service name
- `instance` - Filter by instance name
- `role` - Filter by role name
- `machine` - Filter by machine name

Returns an attribute set where keys are scope identifiers and values are the exported data.

Helper Functions

`clanLib` provides several helper functions for working with exports:

buildScopeKey

Build a scope key string from components:

```
1 clanLib.buildScopeKey {  
2   serviceName = "myservice";  
3   instanceName = "prod";  
4   roleName = "server";  
5   machineName = "backend01";  
6 }  
7 # => "myservice:prod:server:backend01" (internal information)
```

parseScope

Parse a scope string into its components:

```
1 clanLib.parseScope "myservice:prod:server:backend01"  
2 # => {  
3   #   serviceName = "myservice";  
4   #   instanceName = "prod";  
5   #   roleName = "server";  
6   #   machineName = "backend01";  
7 # }
```

getExport

Retrieve a single export by scope:

```
1 clanLib.getExport  
2   { serviceName = "myservice"; machineName = "backend01"; }  
3   exports  
4 # => <some data>
```

selectExports

Filter exports matching specific criteria:

```
1 # Get all exports for a specific service
2 clanLib.selectExports
3   { serviceName = "myservice"; }
4   exports
5 # => {
6 #   <scopes> :: <some data>
7 # }
8
9 # Get all exports for a specific machine
10 clanLib.selectExports
11   { machineName = "backend01"; }
12   exports
13 # => {
14 #   <scopes> :: <some data>
15 # }
```

Export Scope Rules

When defining exports, certain restrictions apply based on context:

1. `perInstance`: Can only export to the matching scope.
2. `perMachine`: Can only export to the machine scope.
3. **Services can only write to:**
 - Their own service scope (e.g., `service = "myservice"`)
 - **Not** other services' scopes

These restrictions prevent accidental conflicts and maintain clear data ownership.

Best Practices

1. **Use the helper functions:** Always use `clanLib.*` functions instead of accessing the internal format directly
2. **Use appropriate scopes:** Export at the most specific scope that makes sense
 - Machine-level data (IPs, hostnames) - export in `perInstance`
 - Instance-level configuration - export

Examples

For real-world use cases, see the [clan-services](#)

Further Reading

- [Service Author Reference](#)
- [Introduction to clanServices](#)
- [Authoring a clanService](#)

← Previous

Authoring a 'clan.service' module

Next →

Internal Services with SSL

Need help? Reach out to [our community](#).